PROTOCOL MANUAL & IMPLEMENTATION REFERENCE

SWP2P

Open-drain parallel peer-to-peer bus for AVR MCUs

Frame fields, edge cycles, state machine, and source-code mapping

Document scope

This manual describes the protocol as implemented by the SWP2P source in this repository. It replaces the earlier v1.0 description where that description differs from the current code. It is an implementation reference, not a claim that every multi-node collision pattern has been hardware-validated.

Document version	v1.1.5 implementation manual
Target implementation	Arduino Uno / ATmega328P, AVR registers and interrupts
Default data width	W4, preset `PRESET_W4_D4_D7`
Default burst capacity	`SWP2P_MAX_BURST = 16`
Wire-format change	DEST(6) + SRC(6) + BURST(1), then LEN or DATA
License	See the repository license file

Contents

1 Purpose and terminology **2**

2 Physical signals and edge convention **2**

3 Frame format **2**
3.1 Logical frame . . . 2
3.2 Burst length encoding . . . 3

4 W4 waveform and named cycles **3**
4.1 Single-byte path . . . 4
4.2 Burst path . . . 4

5 Arbitration and node filtering **4**

6 Clock generation and dynamic rate **4**

7 Implementation map **4**
7.1 State registers . . . 4
7.2 Frame variables and flags . . . 4

8 Application interface and sequence **5**

9 Limits and verification notes **5**

10 Source files **6**

1 Purpose and terminology

SWP2P is a software-defined, multi-node bus. A node can request transmission without a central packet-routing master. One configured node supplies the shared clock; that clock source does not decide which node may transmit. Nodes arbitrate by transmitting the header while BUSY is asserted.

This document distinguishes the *wire protocol* from the *implementation state machine*. Wire fields are the bits visible on DATA lines. State numbers and variable names refer to the current AVR source. Unless stated otherwise, bits are sent most-significant first within each field and each data byte.

Important compatibility note

The v1.1.5 packed header is not wire-compatible with the former two-byte DEST/SRC header. Every node on a shared bus must run a matching implementation. There is no version negotiation or automatic legacy detection.

2 Physical signals and edge convention

Table 1: Signals used by the Uno implementation

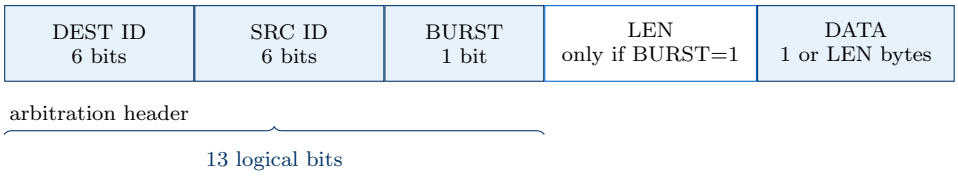
Signal	Uno pin	Direction	Implementation meaning
CLK input	D2 / PD2 / INT0	All nodes receive	Logical-change interrupt. The handler checks the pin level: rising edge is the transmit/update phase; falling edge is the receive/readback phase.
CLK output	D9 / PB1 / OC1A	Clock-source node	Timer1 CTC hardware toggles OC1A. Other nodes connect this to D2.
BUSY	D3 / PD3 / INT1	Wired shared line	Active LOW. A transmitter asserts by driving LOW and releases by switching to input/high impedance.
ACK	D8 / PB0	Wired shared line	Active LOW. The addressed receiver asserts ACK after receiving the last data byte; it releases ACK when BUSY returns idle.
DATA	Preset dependent	Shared parallel bus	Open-drain style: a node sends 0 by driving LOW and sends 1 by releasing the pin.

DATA, BUSY, and ACK use direction-register changes to drive LOW or release the line; external pull-ups establish HIGH. The clock output is generated by the Uno Timer1 OC1A output, not by the DATA open-drain helper. The source configures D2 and D3 for interrupt observation and does not configure the bus pull-ups.

For each wire chunk, the transmitter updates DATA at a rising CLK edge. Receivers sample DATA at the corresponding falling edge. The sender also reads DATA back on falling edges while arbitrating. Consequently, one complete CLK period carries one preset-width chunk, with an update half-cycle and a sample half-cycle.

3 Frame format

3.1 Logical frame



The wire header is the concatenation

$$\boxed{\text{DEST}[5:0]}\ \boxed{\text{SRC}[5:0]}\ \boxed{\text{BURST}}.$$

The header is padded with zero bits at its end to fill an integer number of preset-width chunks. There is no separate start byte: BUSY becoming active starts a frame, and the configured width determines the chunk boundaries.

Table 2: Header fields and encoding

Field	Width	Meaning
DEST	6 bits	Destination node ID, 0–62. Value 63 (`0x3F`) is reserved for broadcast.
SRC	6 bits	Sender node ID, 0–62. The local constructor masks to six bits and maps local ID 63 to 62.
BURST	1 bit	0 means one DATA byte follows directly; 1 means LEN then LEN DATA bytes follow.
Header padding	0–7 bits	Trailing zero padding used only to complete the last preset chunk; not a field.
LEN	compile-time width	Present only for BURST=1. Encodes `len - 2`; bit width depends on `SWP2P_MAX_BURST`.
DATA	8 bits per byte	Data is sent MSB first, chunked according to the selected preset width.

Broadcast is a destination, not a source identity. A broadcast packet is accepted into the FIFO by every receiver, and every receiver that reaches the ACK state drives the shared ACK line LOW. The sender can observe that the shared ACK line was asserted, but cannot identify which receiver acknowledged.

3.2 Burst length encoding

`sendBurst()` uses the burst header bit only when `len > 1`. For a burst, the transmitted LEN value is `len - 2`; the receiver adds 2 after decoding. Thus LEN=0 represents two bytes. LEN width is the minimum width sufficient for the configured maximum and is rounded up to whole DATA chunks.

With the default maximum of 16 bytes, values 0 through 14 are needed, so `LEN_BITS=4`. For W4 this is one nibble and there is no LEN padding. The legal wire burst length is 2–16 bytes; a one-byte call through `sendBurst()` follows the single-byte path and has BURST=0.

Table 3: Default frame chunk counts (`SWP2P_MAX_BURST=16`)

Preset width	Header bits on wire	Header clocks	LEN bits / clocks	Clocks per DATA byte	Notes
W1	13	13	4 / 4	8	Header has no padding.
W2	14	7	4 / 2	4	Header has one trailing pad bit.
W4	16	4	4 / 1	2	Header has three trailing pad bits.
W8	16	2	4 / 1	1	Header has three trailing pad bits.

The W4 header still occupies four clocks, just like the former 16-bit header rounded to four nibbles. The compact LEN saves one clock per burst at the default maximum compared with sending an 8-bit LEN over W4 (two clocks). This is a wire-cycle comparison, not a guarantee of a higher safe CLK frequency.

4 W4 waveform and named cycles

The following diagrams show the logical W4 bus sequence. Each labeled chunk is one full CLK cycle: the value is driven/updated on CLK rising and sampled on CLK falling. For illustration, the header in the burst diagram is DEST=`0x02`, SRC=`0x01`, BURST=1. The packed 13 bits plus three pad bits form `0x0818`, sent as nibbles `0, 8, 1, 8` (bit string `0000 1000 0001 1000`).

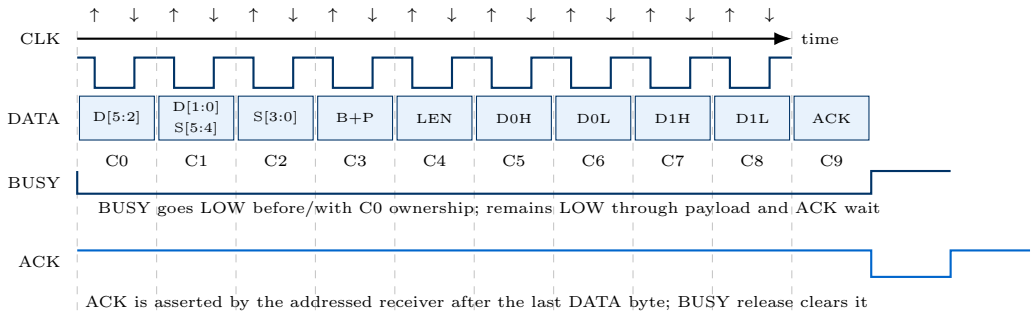


Figure 1: Burst transfer, W4, default 16-byte maximum. C0–C3 are the padded arbitration header; C4 is LEN; each data byte takes two cycles. D=DEST, S=SRC, B=BURST, P=three trailing pad bits; D0H/D0L are the upper/lower nibbles of DATA[0]. The diagram shows two example data bytes, not a fixed frame length.

The fourth header nibble is BURST in its most-significant bit followed by three zero pad bits. In C1, the first two bits complete DEST and the remaining two begin SRC. Chunk boundaries do not have to align with field boundaries.

Cycle name	Physical event	State/variable action in the current source
S	BUSY falls; on the next clock rising edge sender asserts BUSY and drives first header chunk C0.	TX: `_txState=1` (PENDING) becomes 2 (ARB); `_txArbReg`, `_arbMyChunk`, `_arbChunkCount` initialized. RX starts from INT1 in `_onBusyEdge()`.
C0...C(A-1)	Header/arbitration chunk. Rising edge updates DATA; falling edge reads it back and receivers accumulate it.	TX compares `_arbMyChunk` with `_readDataChunk()`; RX accumulates into `_rxAddrByte:_rxSrcByte`. `A=ADDR_CHUNKS`.
L0...L(K-1)	Burst only: LEN chunks.	TX state 3 uses `_txDataReg`, `_txDataChunkCount`; RX state 3 accumulates `_rxDataByte`, then sets `_rxLen`. `K=LEN_CHUNKS`.
D(i,0)...	Data byte i, split into `8/DATA_WIDTH` chunks.	TX state 4 advances `_txIdx`; RX state 4 assembles `_rxDataByte`, then `_fifoPush()` stores data and source.
ACK	Receiver asserts ACK LOW after its final byte; sender waits a bounded number of CLK edges.	RX state 5 drives PB0/D8. TX state 7 reads ACK and clears `FLAG_ACK_FAILED` or sets timeout.
IDLE	Sender releases BUSY; line returns HIGH.	INT1 idle path clears busy, releases ACK, resets RX parser; TX clears sending state.

4.1 Single-byte path

For a single-byte transfer, BURST=0 and LEN is absent. In W4 the wire chunk order is the four header nibbles, then DATA[7:4], then DATA[3:0], followed by ACK handling. A single byte is fixed-length, so the receiver does not need LEN to find its end.

4.2 Burst path

For a burst, BURST=1 and the receiver first reads the configured LEN chunks. After decoding `_rxLen = LEN + 2`, it receives exactly that many data bytes, then moves to ACK. There is no per-byte ACK. The FIFO receives each complete byte as it arrives; the application's `readBytes()` calls can therefore observe partial FIFO contents while a frame is still arriving.

5 Arbitration and node filtering

While multiple senders contend, each sends its header chunks and reads the shared DATA bus back. A sender that detects that it attempted to release a DATA line but observed LOW releases DATA and BUSY, enters TX_LOST, and retries when BUSY returns idle. The receiver side continues parsing the winning header when necessary so a losing node can also receive the winner's packet if addressed to it.

Non-clock-master nodes stop their RX parser once enough header bits identify a non-matching DEST. They do not electrically disconnect: the DATA bus is wired open-drain, and a released line does not interfere with other nodes. The clock-source node continues tracking a non-addressed frame's LEN/data chunk count so it knows when to restore the arbitration clock rate.

Implementation limitation

The code compares multi-bit preset chunks using a bit-mask mismatch test. This is straightforward for W1. For W2/W4/W8 simultaneous collisions, mixed per-line arbitration can produce cases that require a whole-chunk lexicographic comparison to guarantee that exactly one sender wins. This manual records the current implementation; do not treat all simultaneous multi-node arbitration patterns as verified until that behavior is fixed or validated on hardware.

6 Clock generation and dynamic rate

The node started with `begin(true, clkFreq, dataFreq)` generates CLK using Timer1 CTC and OC1A (D9). Other nodes use `begin(false)` and follow the external clock on D2. The default base rate is 100 kHz if omitted; the example in the project uses 45 kHz arbitration and 50 kHz payload.

The source calls `_clkGoFast()` once the packed header ends and the payload section begins. For a burst, that means the LEN phase and DATA run at `dataFreq`; for a single frame DATA runs at `dataFreq`. After the final payload, `_clkGoSlow()` restores `clkFreq` for ACK/idle. Only the clock-source node changes Timer1. Frequency values are not proof of a reliable maximum; ISR execution time and electrical settling still bound practical rates.

7 Implementation map

7.1 State registers

The state values are stored directly in ATmega328P General Purpose I/O Registers through macros: `_txState` is `GPIOR1`; `_rxState` is `GPIOR2`. State transitions occur inside `_onClkEdge()` and `_onBusyEdge()`.

Table 4: TX and RX states in `SWP2P.h`

TX	Name	Meaning	RX	Name	Meaning
0	IDLE	No pending send	0	IDLE/FILTERED	No parser work, including filtered non-destination
1	PENDING	Await start/clock edge	1	HEADER	Packed DEST/SRC/BURST collection
2	ARB	Send/compare header chunks	3	LEN	Receive burst length
3	LEN	Send burst length	4	DATA	Receive addressed DATA
4	DATA	Send payload bytes	5	ACK	Assert ACK after completed receive
6	LOST	Arbitration lost; retry at idle	6	DISCARD	Clock-master counts non-addressed DATA to frame end
7	WAIT_ACK	Await ACK or timeout			

7.2 Frame variables and flags

Table 5: Source names mapped to protocol meaning

Name	Storage	Role
`_nodeId`	static byte	Local source ID, normalized to 0–62 by constructor.
`_txDestId`	static byte	DEST in low six bits; bit 7 is an internal marker used by `sendBurst()`, not a wire header byte.
`_txLen`, `_txIdx`	static bytes	Requested payload length and current TX byte index.
`_txBuffer[]` / `_txData`	static bytes	Copied burst payload / single-byte payload. `sendBurst()` copies before returning.
`_txArbReg`	16-bit	Left-aligned packed 13-bit header plus trailing preset padding, shifted out MSB-first.
`_arbChunkCount`	byte	Remaining header chunks after first chunk is driven.
`_arbMyChunk`, `_arbChunksSent`	bytes	Local chunk used for readback arbitration and number of chunks driven.
`_txDataReg`, `_txDataChunkCount`	byte	Shift register and remaining LEN/DATA chunks.
`_rxAddrByte`, `_rxSrcByte`	bytes	Together hold the 16-bit packed-header receive accumulator. After decode, `_rxSrcByte` is reused for source ID.
`_rxDataByte`	byte	LEN accumulator or current DATA byte accumulator.
`_rxLen`, `_rxByteIdx`, `_rxChunkCount`	bytes	Decoded data byte count, current byte index, chunks remaining.
`_rxFifo[]`, `_rxSrcFifo[]`	FIFO	Parallel data and sender-ID queues, indexed in lockstep.
`GPIOR0` flags	register	Sending, busy, addressed-to-me, and ACK-failed flags (bits 0, 1, 3, 5).
`_ocrArb`, `_ocrData`	16-bit	Timer1 compare values for base and payload rates.

Header encoding in the TX path is equivalent to

```
header = (DEST << 7) | (SRC << 1) | BURST;
wireHeader = header << (16 - 13); // top-align, then emit preset-width chunks
```

The receiver collects the padded wire header and shifts right by `ADDR_PADDING_BITS` before extracting DEST, SRC, and BURST.

8 Application interface and sequence

Table 6: Common public API

API	Behavior
`begin(clkIsOutput, clkFreq, dataFreq)`	Initializes pins, interrupts, and optional Timer1 clock output/rates.
`send(dest, byte)`	Queues one-byte frame (BURST=0); non-blocking.
`sendBurst(dest, ptr, len)`	Copies data into TX storage and queues a frame; len=1 uses single-byte format.
`sendBurst(dest, buffer)`	Queues the current `SWP2PBuffer` contents.
`available()` / `read()`	Check/read one data byte plus its source ID.
`readBytes(data, src, max)`	Drains currently available FIFO bytes; it does not wait for whole frame completion.
`isSending()` / `isBusy()`	Report local send-in-progress and shared BUSY status.
`isAckFailed()`	Returns and clears the most recent ACK-timeout flag.

1. Prepare data in application memory.
2. Call `send()` or `sendBurst()`. A `true` return means accepted into the local TX state machine, not yet delivered.
3. The library arbitrates, sends the frame, and waits for ACK. The application can continue running while the ISR advances the frame.
4. Check `isSending()` until it clears, then call `isAckFailed()` to inspect the result.
5. On the receiver, drain data and source IDs from the FIFO. A FIFO has depth `SWP2P_FIFO_DEPTH` (default 16); it is a byte queue, not a packet queue.

9 Limits and verification notes

- `SWP2P_MAX_BURST` is configurable from 1 to 255; it sets the TX storage capacity and compile-time LEN width. The receiver rejects decoded lengths above this configured limit.
- Node IDs 0–62 are usable; 63 is reserved as broadcast DEST. All nodes on a bus must agree on preset, frame version, maximum burst setting, and compatible clock behavior.
- There is no CRC/checksum in the frame. ACK confirms that the receiver reached the end of its receive path, not that every data bit was correct.
- The RX FIFO is byte-oriented and finite (default 16 entries). `_fifoPush()` drops a byte silently if the FIFO is full; applications should drain it promptly.

- The manual's diagrams describe logical edge order. Exact rise/fall time, maximum reliable frequency, and simultaneous arbitration behavior require the actual board, wiring, pull-ups, and source build to validate.
- On Uno, D9 is Timer1 OC1A and D8 is PB0/ACK in this source. A clock-source node cannot also use a preset that assigns D9 as DATA. D2 and D3 are fixed CLK/BUSY pins. The example's D10 input is separate from W4 D4–D7.

10 Source files

- `src/SWP2P.h`: public API, TX/RX state machines, ISR handlers, timer-rate transitions.
- `src/SWP2Ppreset.h`: preset pin mappings, open-drain DATA operations, frame-width traits.
- `src/SWP2P.cpp`: static storage, FIFO implementation, Timer1 setup.
- `src/SWP2Pbuffer.h`: application-side fixed-capacity buffer helper.